



Yeditepe University

CSE351: Programming Languages

Term Project Handout

Prepared by: **Asst. Prof. Gizem Süngü Terci**

In this project, you will design and implement YU Assembly, a custom pseudo-assembly language. The project consists of the following two main tasks:

- Design the YU Assembly Language.
- Optimize execution performance by applying loop unrolling to the code.

The instruction set of YU Assembly is given in Table 1.

Instruction	Syntax	Description
mov	mov Rd, imm	Load immediate value into register Rd
load	load Rd, mem[addr]	Load from memory address
store	store Rs, mem[addr]	Store register Rs to memory address
addi	addi Rd, Rs, imm	$Rd = Rs + imm$
add	add Rd, Rs1, Rs2	$Rd = Rs1 + Rs2$
and	and Rd, Rs1, Rs2	$Rd = Rs1 \& Rs2$
or	or Rd, Rs1, Rs2	$Rd = Rs1 Rs2$
xor	xor Rd, Rs1, Rs2	$Rd = Rs1 \wedge Rs2$
blt	blt Rs1, Rs2, label	Branch if $Rs1 < Rs2$
bgt	bgt Rs1, Rs2, label	Branch if $Rs1 > Rs2$
beq	beq Rs1, Rs2, label	Branch if $Rs1 == Rs2$

Table 1: YU Assembly Instruction Set

The following program copies 50 values from memory locations [0..49] to memory locations [100..149]:

```

1 mov R2, 0
2 mov R3, 100
3 mov R4, 50
4 L: load R1, mem[R2]
5     store R1, mem[R3]
6     addi R2, R2, 1
7     addi R3, R3, 1
8     blt R2, R4, L

```

Task 1: Write a complete BNF grammar that formally defines the syntax of YU Assembly.

Task 2: Implement a lexical analyzer (Flex) and a syntax analyzer (Bison/Yacc) for YU Assembly.

Task 3: Loop unrolling is an optimization technique that reduces loop overhead by duplicating the loop body. Unrolling by a factor of 2 replaces each iteration with two iterations per loop branch, reducing the number of branch and increment instructions. Loops must be detected using backward branch detection. A loop is defined as a sequence of instructions containing exactly one backward branch instruction whose target label matches a previously defined label in the program.

In this task, you are expected to implement loop unrolling for programs written in YU Assembly.

For example, the sample program above becomes the following after loop unrolling:

```

1 mov R2, 0
2 mov R3, 100
3 mov R4, 50
4 L: load R1, mem[R2]
5     store R1, mem[R3]
6     load R1, mem[R2+1]
7     store R1, mem[R3+1]
8     addi R2, R2, 2
9     addi R3, R3, 2
10    blt R2, R4, L

```

YU Assembly Language Rules

Ensure that your implementation satisfies the following requirements:

- **1.** Every instruction in the language must be supported and accepted by your lexer and parser without errors.
- **2.** The parser must support multiple labeled blocks and branch instructions referencing different labels.

- **3.** Memory addressing formats such as `mem[REG]`, `mem[REG+NUMBER]`, and `mem[REG-NUMBER]` must be supported.
- **4.** Programs may include chained `and`, `or`, and `xor` instructions. Programs without loops must also be accepted.
- **5.** Only registers `R0`{`R7` are valid. Using `R8` or higher must produce an error.
- **6.** The `mov` instruction requires a register as its first operand. Using a label instead must produce a syntax error.
- **7.** Missing commas must be detected as syntax errors.
- **8.** All instructions must strictly follow the defined syntax. Any deviation must result in an error.

Grading

Component	Points
BNF Grammar	15
Lexer (Flex)	15
Parser (Bison/Yacc)	20
Loop Unrolling	35
Report (Design and Analysis)	15
Total	100
Bonus	+20

Table 2: Project Grading Breakdown

Submissions that rely solely on string manipulation instead of structured parsing (AST-based or equivalent) may receive reduced credit. AST (Abstract Syntax Tree) refers to the structured representation of the program produced by the parser.

Deliverables

The following files should exist in a zip file named `Surname_CSE351_Project.zip`.

- A BNF grammar document describing the YU Assembly syntax
- A Flex lexical analyzer file (`.l`)
- A Bison/Yacc parser file (`.y`)

- Scripts or command line statements to compile and run your project
- Three valid and three invalid sample input programs demonstrating that your implementation covers the specified language rules
- A report explaining how you designed and implemented your project in detail.

Bonus (+20 pts)

You may implement one of the following compiler optimizations:

- Implement a symbol table to validate that all branch targets refer to declared labels (semantic analysis with label resolution).
- Constant folding of arithmetic expressions (compile-time evaluation of constant operations)
- Dead code elimination (removal of instructions whose results are never used)
- Peephole optimization on small instruction windows (2–3 instructions)

The bonus evaluation will consider correctness, completeness, and integration with the AST-based implementation.